

Universal Progress Monitoring Package

A Domain-Independent Framework for Monitoring Intelligence
Growth,
State, Trajectory, Intent Preservation, and Drift Detection

Version 1.0.0
HAEL Foundation
June 2026

Table of Contents

1. Introduction	3
1.1 Core Design Principles	3
1.2 The Problem with Conventional Monitoring	4
2. Architecture Overview	4
3. The 15 Layers - Detailed Specification	5
3.1 Layer 1: Intent Model	5
3.2 Layers 2-3: State Representations	5
3.3 Layer 4: Goal Distance Metric	6
3.4 Layer 5: Progress Score	6
3.5 Layer 6: Trajectory Quality	7
3.6 Layer 7: Intent Preservation	7
3.7 Layer 8: Context Preservation	7
3.8 Layers 9-10: Velocity and Acceleration	8
3.9 Layer 11: State Health	8
3.10 Layer 12: Drift Detection	8
3.11 Layer 13: Future Projection	9
3.12 Layer 14: Quality Assessment	9
3.13 Layer 15: Intervention Engine	10
4. Intelligence Training Stage Model	10
5. The Unified Monitor Report	11
6. API Reference and Usage	12
6.1 Basic Usage	12
6.2 Context Tracking	13
6.3 Stage Assessment	13
6.4 Serialization and Persistence	13
6.5 Visualization	13
7. Package Structure	14
8. Alignment with the Growth and Maturation Protocol	14
9. Extension and Customization	15
10. Mathematical Foundations	15
10.1 Euclidean Distance (Layer 4)	15
10.2 Progress Score (Layer 5)	15
10.3 Cosine Similarity (Layer 6)	15
10.4 Velocity and Acceleration (Layers 9-10)	16
10.5 Completion Probability	16

1. Introduction

The Universal Progress Monitoring Package (UPMP) is a domain-independent framework designed to monitor the growth, state, and trajectory of intelligence systems. Unlike conventional training monitors that track only accuracy or loss, UPMP measures whether an intelligence is still pursuing its original objective, whether it has lost important context, whether its growth trajectory is aligned with its desired direction, and whether the quality of its progress is sufficient. These dimensions are critical for any system that must mature toward a specific goal without drifting, forgetting, or degrading along the way.

The framework was conceived to address a fundamental gap in intelligence monitoring: most systems measure how much progress has been made, but very few measure whether that progress is correct. A system can achieve 80% progress toward a goal while simultaneously drifting away from its original intent, losing critical context, and developing capabilities that do not serve the intended purpose. UPMP makes these hidden dimensions visible and actionable.

UPMP is designed to be completely domain-independent. It can monitor AI training processes, human learning systems, workflow optimization, agent maturation, organizational capability development, or any process where growth toward a defined goal must be tracked with fidelity. The package is implemented as a reusable Python library with a clean API that requires only three inputs: an intent definition, a current state, and a desired state.

1.1 Core Design Principles

The UPMP framework is built on several foundational principles that distinguish it from conventional monitoring approaches. First, it separates progress, state, intent preservation, and quality into independent dimensions. This separation is crucial because a system can be making rapid progress while simultaneously drifting from its original purpose. Second, everything is measured against the original intent. This prevents goal drift by making deviation from the stated purpose explicit and quantifiable. Third, the framework provides not just measurement but intervention. When drift, stagnation, or quality degradation is detected, the system generates specific, actionable recommendations for correction.

Principle	Description
Intent-First	Everything is measured against the original intent to prevent goal drift
Multi-Dimensional	Progress, trajectory, context, quality, and intent are measured independently
Domain-Independent	No assumptions about the type of intelligence or training process
Actionable	Every detected issue comes with a specific intervention recommendation
Reusable	A single package that can be applied to any monitoring scenario
Composable	Custom dimensions, weights, and thresholds allow flexible configuration

1.2 The Problem with Conventional Monitoring

Conventional monitoring systems in AI and machine learning focus almost exclusively on quantitative metrics: accuracy, loss, reward, BLEU score, or other task-specific measures. While these metrics tell you how much a system has learned, they fail to capture several critical dimensions. They do not tell you whether the system is still pursuing the original objective or has drifted to a related but different goal. They do not reveal whether important context has been forgotten. They cannot detect when progress is being made in the wrong direction. And they provide no mechanism for understanding whether the quality of learning is sufficient, or whether the system is achieving high scores through shortcut strategies rather than genuine understanding.

This gap becomes particularly dangerous in the context of intelligence systems that are intended to mature over extended periods. The longer the training process, the more opportunity there is for subtle drift to accumulate, for context to erode, and for the system to develop behaviors that satisfy surface-level metrics while missing the deeper purpose. UPMP was designed to make these invisible risks visible and manageable.

2. Architecture Overview

The UPMP framework is organized into 15 layers, each addressing a distinct aspect of progress and state monitoring. These layers are not independent silos; they feed into each other and produce a unified monitoring report that gives a comprehensive picture of the intelligence system's health, trajectory, and likelihood of reaching its goal.

Layer	Name	Core Question
1	Intent Model	Captures the why of the system
2	State Representation	Where is the system now?
3	Desired State	Where should the system be?
4	Goal Distance Metric	How far from the goal?
5	Progress Score	What percentage achieved?
6	Trajectory Quality	Is the direction correct?
7	Intent Preservation	Is the original goal still pursued?
8	Context Preservation	Has important context been lost?
9	Growth Velocity	How fast is the system growing?
10	Acceleration	Is growth speeding up or slowing down?

11	State Health	Composite health score
12	Drift Detection	Are there deviations from the path?
13	Future Projection	Where will the system be in the future?
14	Quality Assessment	Is the progress of good quality?
15	Intervention Engine	What corrective actions are needed?

The layers form a processing pipeline: Intent (Layer 1) anchors everything. States (Layers 2-3) define the current and target positions. Distance and Progress (Layers 4-5) measure the gap. Trajectory and Intent Preservation (Layers 6-7) ensure the direction is correct. Context Preservation (Layer 8) ensures nothing important has been lost. Velocity and Acceleration (Layers 9-10) measure the dynamics of growth. State Health (Layer 11) produces a composite score. Drift Detection (Layer 12) identifies deviations. Future Projection (Layer 13) forecasts likely outcomes. Quality Assessment (Layer 14) evaluates the merit of progress. And the Intervention Engine (Layer 15) generates actionable recommendations when issues are detected.

3. The 15 Layers - Detailed Specification

3.1 Layer 1: Intent Model

The Intent Model is the foundational layer of the UPMP framework. It captures the why of the intelligence system, defining the purpose, success criteria, and constraints that the system must satisfy. Everything that follows in the monitoring pipeline is measured against this intent. Without a clearly defined intent, it is impossible to detect drift, assess trajectory quality, or determine whether progress is meaningful. The intent serves as the anchor point that prevents the system from gradually shifting to a different goal without anyone noticing.

The Intent data structure includes a unique identifier, a human-readable description, a priority level (low, medium, high, or critical), a list of measurable success criteria that define what it means for the intent to be achieved, and a list of constraints that bound the system's behavior. The success criteria are particularly important because they form the basis for intent preservation scoring in Layer 7. If success criteria are not being actively pursued, the system has drifted from its original purpose.

3.2 Layers 2-3: State Representations

Layers 2 and 3 define the current and desired states of the intelligence system. The current state captures where the system is right now, while the desired state captures where it needs to be. Both use the same dimensional structure, with each dimension normalized to the range [0.0, 1.0]. The default dimensions cover six core intelligence facets: knowledge (factual and conceptual understanding), skills (applied proficiency), memory (retention and recall), reasoning (logical and analytical capability), performance

(task execution quality), and confidence (self-assessment accuracy and calibration). Additionally, a custom dictionary allows arbitrary domain-specific dimensions to be added, making the framework adaptable to any monitoring context.

The normalized range is critical for the mathematical operations performed in subsequent layers. It enables Euclidean distance calculation (Layer 4), progress scoring (Layer 5), and trajectory analysis (Layer 6) to operate consistently regardless of the domain or the specific dimensions being monitored. All values are validated to ensure they fall within the [0.0, 1.0] range, and the framework raises an error if any value is out of bounds.

Dimension	Description	Range
knowledge	Factual and conceptual understanding	0.0 - 1.0
skills	Applied skill proficiency	0.0 - 1.0
memory	Retention and recall ability	0.0 - 1.0
reasoning	Logical and analytical capability	0.0 - 1.0
performance	Task execution quality	0.0 - 1.0
confidence	Self-assessment accuracy	0.0 - 1.0
custom	Arbitrary domain-specific dimensions	0.0 - 1.0

3.3 Layer 4: Goal Distance Metric

The Goal Distance Metric quantifies how far the current state is from the desired state using Euclidean distance. This provides both a raw distance value and a normalized distance in the range [0, 1], where 0 means the system is at its goal and 1 means it is maximally far away. The Euclidean distance formula is $D = \sqrt{\sum((s_i - g_i)^2)}$ where s represents the current state dimensions and g represents the goal state dimensions. The normalized distance divides the raw distance by the maximum possible distance ($\sqrt{n}$ where n is the number of dimensions), providing a scale-independent measure.

This metric is foundational because it gives a single number that captures the overall gap between current and desired performance. However, it is only one piece of the picture. A small distance does not guarantee that the system is on the right trajectory, and a large distance does not necessarily mean the system is failing. The subsequent layers provide the qualitative context that makes the distance metric meaningful.

3.4 Layer 5: Progress Score

The Progress Score calculates what percentage of the target has been achieved using the formula $P = (A / T) \times 100$, where A is the sum of current achieved values and T is the sum of target values across all dimensions. This provides a simple, intuitive measure of how far along the system is toward its goal. A

progress score of 50% means the system has reached roughly half of its target capability.

While the Progress Score is the most immediately intuitive metric, it is also the most dangerous to rely on in isolation. A system can have a high progress score while moving in the wrong direction, while having lost critical context, or while developing capabilities that do not serve the original intent. This is why UPMP measures progress alongside trajectory quality, intent preservation, and quality assessment rather than treating progress as a sufficient indicator of health.

3.5 Layer 6: Trajectory Quality

Trajectory Quality measures whether the direction of growth is correct, not just whether growth is occurring. It computes the cosine similarity between the actual direction vector (the movement from the previous state to the current state) and the desired direction vector (the direction from the current state toward the goal state). A cosine similarity of 1.0 means the system is moving directly toward its goal, 0.0 means it is moving perpendicular to the desired direction, and -1.0 means it is moving directly away from its goal.

The cosine similarity is mapped to a 0-100 score and classified into three alignment categories: **ALIGNED** (score 75-100, the system is moving toward the goal), **PARTIALLY_ALIGNED** (score 40-74, the system is making some progress but not optimally), and **DIVERGENT** (score 0-39, the system is moving away from the goal). This classification provides an immediate qualitative assessment of whether the system's growth is constructive or destructive. A system with high progress but divergent trajectory is making rapid progress in the wrong direction, which is worse than slow progress in the right direction.

3.6 Layer 7: Intent Preservation

Intent Preservation is one of the most critical metrics in the UPMP framework. It measures how much of the original intent is still being actively pursued by the system. The score is calculated as the ratio of actively pursued success criteria to the total number of original success criteria, expressed as a percentage. The system also tracks which goals are missing (no longer being pursued), which are corrupted (being pursued in a distorted form), and which remain active.

This layer addresses a particularly insidious failure mode: goal drift. Goal drift occurs when a system gradually shifts from its original objective to a related but different objective, often without any explicit decision to do so. For example, a system intended to develop deep reasoning might gradually shift to optimizing for speed, or a system intended to build user understanding might shift to maximizing engagement metrics. Intent Preservation makes this drift visible and quantifiable, enabling early correction before the system has fully abandoned its original purpose.

3.7 Layer 8: Context Preservation

Context Preservation tracks how much of the original context has been retained by the intelligence system. Context items are user-defined strings representing important information, preferences,

constraints, or knowledge that the system should not forget. The framework tracks three categories: retained items (still present in the current context), forgotten items (no longer present), and distorted items (present but altered from their original form). The integrity score is calculated as the effective retained ratio, where distorted items count as half-retained.

Context loss is a common problem in long-running intelligence systems. As new information is acquired, older information can be overwritten, deprioritized, or simply lost. This is particularly problematic when the lost context includes foundational assumptions, user preferences, or constraints that the system was supposed to maintain throughout its operation. By explicitly tracking context items and their status, UPMP makes it possible to detect and correct context erosion before it leads to significant behavioral degradation.

3.8 Layers 9-10: Velocity and Acceleration

Growth Velocity measures the rate of state change over time, calculated as the Euclidean magnitude of state change divided by the time elapsed between consecutive evaluation snapshots. This metric tells you how fast the intelligence system is moving toward (or away from) its goal. High velocity with aligned trajectory means rapid progress; high velocity with divergent trajectory means rapid drift.

Acceleration measures the change in velocity over time, calculated from at least three consecutive evaluation snapshots. Positive acceleration indicates that growth is speeding up, which could indicate a breakthrough or an approaching convergence. Negative acceleration indicates that growth is slowing down, which could indicate a plateau, saturation, or the onset of stagnation. Near-zero acceleration combined with near-zero velocity is a strong signal that the system is stuck and needs an intervention such as increased training diversity or more challenging practice scenarios.

3.9 Layer 11: State Health

State Health produces a composite score that combines the key health indicators into a single weighted average. The default weights are: progress (30%), alignment (25%), context (20%), confidence (15%), and consistency (10%). These weights can be customized to reflect the priorities of a specific monitoring context. The composite score provides a quick qualitative assessment: THRIVING (85+), HEALTHY (70-84), MODERATE (50-69), AT_RISK (30-49), or CRITICAL (below 30).

The State Health composite is designed to be the first number a user looks at. If the composite health is high, the system is doing well across all dimensions. If it is low, the user should examine the individual components to determine which dimension is causing the degradation. This two-level diagnostic approach (composite first, details second) makes the monitoring output scannable for busy operators while still providing the depth needed for thorough analysis.

3.10 Layer 12: Drift Detection

Drift Detection identifies specific deviations from the intended path across five categories. Knowledge Drift occurs when a dimension is significantly below its target level. Goal Drift occurs when success

criteria are no longer being actively pursued. Context Drift occurs when context items have been lost. Reasoning Drift occurs when the trajectory is divergent from the desired direction. Behavior Drift occurs when the system appears stagnant with low velocity and non-positive acceleration.

Each detected drift is characterized by its type, severity (low, medium, high, or critical), source (which dimension or component is affected), a numeric score, a human-readable description, and the affected dimensions. The severity is automatically determined based on thresholds: a gap of 50% or more is critical, 30-49% is high, 15-29% is medium, and below 15% is low. This multi-category drift detection ensures that different types of deviation are caught early and attributed correctly, enabling targeted interventions.

Drift Type	Trigger	Detection Method
Knowledge Drift	Dimension below target	Per-dimension gap analysis
Goal Drift	Criteria no longer pursued	Active vs original criteria comparison
Context Drift	Context items lost	Original vs current context set comparison
Reasoning Drift	Divergent trajectory	Cosine similarity analysis
Behavior Drift	System stagnation	Velocity and acceleration analysis

3.11 Layer 13: Future Projection

Future Projection forecasts the system's likely state at 7, 30, and 90 day horizons by extrapolating the current direction of growth. For each horizon, it computes the projected state (where each dimension will be if current trends continue), the projected goal completion percentage, and any projected risks such as dimensions that will stagnate or overshoot their targets. The projection confidence is based on the length of available history, with longer histories producing more reliable projections.

This layer is particularly valuable for long-running intelligence training processes where it is important to anticipate problems before they become critical. If the projection shows that a dimension will never reach its target under current trends, this is a clear signal that the training approach needs to be adjusted. Similarly, if the projection shows that a dimension will overshoot its target, this may indicate that the system is over-optimizing in one area at the expense of others.

3.12 Layer 14: Quality Assessment

Quality Assessment evaluates the merit of the progress being made, not just its quantity. It measures four dimensions: correctness (how close the current state is to the desired state), robustness (how consistent the trajectory has been), consistency (same as robustness, measuring variance in direction), and explainability (based on the system's self-assessment confidence). The overall quality score is a weighted composite of these four dimensions.

This layer addresses the critical distinction between "moving fast" and "learning well." A system with 80% progress but 45% quality is making rapid progress through shallow learning, which is likely to produce brittle capabilities that fail under novel conditions. By contrast, a system with 50% progress and 85% quality is building deep, robust understanding that will serve it well in the long term. The Quality Assessment makes this distinction explicit and actionable.

3.13 Layer 15: Intervention Engine

The Intervention Engine generates specific, actionable recommendations when issues are detected. It maps detected drifts and quality problems to appropriate intervention types: Goal Drift triggers intent re-alignment, Knowledge Drift triggers increased training diversity, Context Drift triggers memory reinforcement, Reasoning Drift triggers complexity reduction, and Behavior Drift triggers increased practice. Additionally, the engine generates interventions for critically low intent preservation, high progress with low quality (rushing without understanding), and low context integrity.

Intervention Type	Trigger	Description
realign_intent	Goal drift or low intent preservation	Re-examine original intent and ensure all criteria are pursued
increase_training_diversity	Knowledge gap in specific dimension	Increase training focus and variety in weak areas
memory_reinforcement	Context items being lost	Implement spaced repetition or context restoration
reduce_complexity	Reasoning trajectory divergent	Simplify current task and re-align with original intent
increase_practice	System stagnation	Introduce more challenging scenarios to stimulate growth
context_restoration	Low context integrity	Restore key context items and reinforce memory strategies

4. Intelligence Training Stage Model

The Intelligence Training Stage Model provides a 10-stage progression framework that maps the maturation of an intelligence system from initialization to goal realization. This model is aligned with the Growth and Maturation Protocol's five stages (Neonate, Learner, Practitioner, Expert, Supreme) and extends them into a more granular 10-stage model that provides finer resolution for monitoring purposes.

Stage	Name	Growth Protocol	Description
0	Initialization	Neonate	Seed state, no knowledge or skills
1	Knowledge Acquisition	Neonate-Learner	Absorbing raw information, high uncertainty
2	Pattern Formation	Learner	Recognizing patterns, forming connections
3	Reasoning Development	Learner-Practitioner	Active reasoning, anticipation emerges
4	Generalization	Practitioner	Transferring knowledge across domains
5	Self-Evaluation	Practitioner-Expert	Metacognition, recognizing weaknesses
6	Self-Improvement	Expert	Actively modifying own processes
7	Autonomous Optimization	Expert-Supreme	Self-optimizing without external guidance
8	Intent Mastery	Supreme	Full intent internalization, zero drift
9	Goal Realization	Supreme (fulfilled)	Goal achieved as originally intended

The stage assessment algorithm determines the current stage by finding the highest stage for which the system meets all minimum threshold values across key dimensions. Each stage has a defined profile of minimum expected values, and the system qualifies for a stage only when it exceeds all of those thresholds. The assessment also calculates the distance to the next stage and the overall distance to the final goal realization stage, providing a roadmap for development priorities.

5. The Unified Monitor Report

All 15 layers feed into a single unified MonitorReport object that provides a comprehensive snapshot of the intelligence system's current status. The report contains composite scores (intent score, progress score, state health, trajectory quality, context integrity, drift score, velocity, acceleration, quality score, and completion probability), state snapshots (current, desired, and projected), detailed sub-reports (intent preservation, trajectory, context, drifts, quality), intervention recommendations, distance metrics, and metadata. The report includes a summary() method that produces a human-readable text overview and a status_label() method that returns a qualitative status (THRIVING, HEALTHY, MODERATE, AT_RISK, or CRITICAL).

Field	Range	Description
intent_score	0-100	How well the intent is being pursued

progress_score	0-100	Raw progress toward the goal
state_health	0-100	Composite health score
trajectory_quality	0-100	Direction alignment
context_integrity	0-100	How much context is retained
drift_score	0-100	How much drift has occurred
velocity	Float	Rate of state change per day
acceleration	Float	Change in velocity per day
quality_score	0-100	Quality of progress
completion_probability	0-100	Likelihood of reaching the goal

6. API Reference and Usage

The UPMP package is designed for minimal setup and maximum flexibility. The core workflow requires only three steps: define the intent, set the states, and call `evaluate()`. The following code demonstrates the most basic usage pattern, showing how to create a monitor, configure it with an intent and state pair, and produce a comprehensive evaluation report.

6.1 Basic Usage

```

from upmp import UPMPMonitor, Intent, State, DesiredState
from upmp.models import Priority

intent = Intent(
    id="math-001",
    description="Learn mathematics",
    priority=Priority.HIGH,
    success_criteria=["solve calculus", "explain concepts"],
)

monitor = UPMPMonitor(intent=intent)

current = State(knowledge=0.45, reasoning=0.30, memory=0.70)
desired = DesiredState(knowledge=0.90, reasoning=0.90, memory=0.90)

monitor.set_states(current, desired)
report = monitor.evaluate()
print(report.summary())

```

6.2 Context Tracking

Context tracking allows you to monitor whether important information is being retained throughout the intelligence training process. You set the original context items, optionally specify which items are currently retained, and which have been distorted. The framework will then compute context integrity and detect context drift automatically.

```
monitor.set_context(  
    original=["Calculus fundamentals", "Algebra prerequisites",  
            "Logic and proof methods"],  
    current=["Calculus fundamentals", "Algebra prerequisites"],  
    distorted=[],  
)
```

6.3 Stage Assessment

The stage assessment feature allows you to determine where the intelligence currently stands in its maturation journey. It returns a StageAssessment object containing the current stage, confidence level, distance to the next stage, and distance to the final goal realization stage.

```
stage = monitor.assess_stage()  
print(f"Stage: {stage.stage_name}")  
print(f"Confidence: {stage.confidence:.1f}%")  
print(f"Distance to next: {stage.distance_to_next:.1f}%")
```

6.4 Serialization and Persistence

Reports can be serialized to JSON for storage, transmission, or further analysis. The UPMPSerializer class provides methods for converting reports to and from JSON format, as well as saving and loading from files.

```
from upmp.serializers import UPMPSerializer  
  
serializer = UPMPSerializer()  
json_str = serializer.to_json(report)  
serializer.save_json(report, "report.json")  
loaded = serializer.load_json("report.json")
```

6.5 Visualization

The UPMPVisualizer class generates professional charts from monitor reports, including radar charts (current vs desired state), progress bar charts, health gauge visualizations, trajectory over time charts, and a composite dashboard that combines all of these into a single comprehensive view. Visualizations

are saved as PNG images suitable for embedding in reports or displaying in dashboards.

```
from upmp.visualizer import UPMPVisualizer

viz = UPMPVisualizer(report)
viz.radar_chart("state_radar.png")
viz.progressBars("progress.png")
viz.health_gauge("health.png")
viz.composite_dashboard("dashboard.png")
```

7. Package Structure

The UPMP package is organized into a clean module structure that separates concerns and makes it easy to extend or customize individual components. The core monitor is in the monitor module, data models are in the models module, the stage system is in the stages module, visualization is in the visualizer module, and serialization is in the serializers module.

Module	Description
upmp/__init__.py	Package entry point, version, public API exports
upmp/models.py	Core data models (Intent, State, DesiredState, MonitorReport, etc.)
upmp/monitor.py	UPMPMonitor class implementing all 15 layers
upmp/stages.py	Intelligence Training Stage Model (10 stages)
upmp/visualizer.py	Chart generation (radar, bars, gauge, dashboard)
upmp/serializers.py	JSON serialization and deserialization

8. Alignment with the Growth and Maturation Protocol

The UPMP framework is designed to be fully compatible with the Growth and Maturation Protocol described in the Supreme Intelligence Framework. The Protocol defines five stages of dyadic maturation (Neonate, Learner, Practitioner, Expert, Supreme), each characterized by specific capabilities, behaviors, and memory configurations. The UPMP 10-stage model maps directly to these five stages while providing finer resolution for monitoring purposes.

The Daily Consolidation Ritual described in the Growth Protocol, which involves reviewing episodic memory, extracting semantic patterns, updating the user model, pruning low-value memories, and verifying uncertain claims, maps directly to several UPMP monitoring layers. The review step

corresponds to Context Preservation (Layer 8), the extraction step corresponds to Trajectory Quality (Layer 6), the update step corresponds to State Health (Layer 11), the pruning step corresponds to Drift Detection (Layer 12), and the verification step corresponds to Quality Assessment (Layer 14).

This alignment means that UPMP can serve as the monitoring backbone for Supreme Intelligence twins, providing real-time visibility into their maturation process while remaining fully compatible with the existing Growth Protocol architecture. The monitor can be embedded within the Daily Consolidation Ritual to produce a comprehensive health report at each consolidation cycle, ensuring that any drift, context loss, or quality degradation is detected and corrected before it compounds over time.

9. Extension and Customization

The UPMP framework is designed to be extended and customized for specific domains and use cases. Custom state dimensions can be added through the custom dictionary on State and DesiredState objects. Custom health weights can be configured when creating a UPMPMonitor to reflect domain-specific priorities. Custom drift thresholds can be set per dimension to adjust sensitivity. The stage model's profile thresholds can be modified to reflect domain-specific expectations for each stage.

For advanced customization, the UPMPMonitor class is designed with clean internal separation, making it straightforward to override individual layer computations. For example, a domain-specific monitor might replace the trajectory quality algorithm with one that accounts for domain-specific constraints, or replace the intervention engine with one that has access to domain-specific corrective actions. The framework provides the skeleton and the sensible defaults; the user provides the domain expertise.

10. Mathematical Foundations

The UPMP framework relies on several mathematical formulations that provide rigor and consistency to its measurements. This section summarizes the key formulas used across the 15 layers.

10.1 Euclidean Distance (Layer 4)

The goal distance between the current state s and the desired state g is computed as the Euclidean distance: $D = \sqrt{\sum((s_i - g_i)^2)}$ for $i = 1$ to n , where n is the number of dimensions. The normalized distance is $D_{\text{norm}} = D / \sqrt{n}$, providing a value in $[0, 1]$.

10.2 Progress Score (Layer 5)

Progress is calculated as $P = (A / T) \times 100$, where $A = \sum(s_i)$ is the sum of current values and $T = \sum(g_i)$ is the sum of target values across all dimensions.

10.3 Cosine Similarity (Layer 6)

Trajectory alignment is measured using cosine similarity: $\cos(\theta) = (d \cdot g) / (\|d\| \times \|g\|)$, where d is the direction vector (current minus previous state) and g is the desired direction vector (goal minus current state). The cosine similarity ranges from -1 (perfectly opposite) to 1 (perfectly aligned) and is mapped to a 0-100 score using the formula $\text{score} = ((\cos + 1) / 2) \times 100$.

10.4 Velocity and Acceleration (Layers 9-10)

Growth velocity is $V = \Delta S / \Delta t$, where ΔS is the Euclidean magnitude of state change and Δt is the time elapsed in days. Acceleration is $a = (V_2 - V_1) / \Delta t$, computed from at least three consecutive evaluation snapshots.

10.5 Completion Probability

The completion probability estimates the likelihood of reaching the desired goal using a weighted combination: $P_{\text{complete}} = 0.40 \times \text{Progress} + 0.25 \times \text{Intent} + 0.20 \times \text{Quality} + 0.15 \times \text{InverseDrift}$, where $\text{InverseDrift} = 100 - \text{DriftScore}$. This formula reflects the principle that reaching a goal requires not just progress, but aligned intent, quality learning, and minimal drift.